Neurosymbolic Routing for Reliable Reasoning on Resource-Constrained Edge Devices

Anonymous

***Abstract*—Small language models (SLMs) deployed on edge devices suffer from poor accuracy on structured reasoning tasks such as arithmetic, algebra, and formal logic. We present a neurosymbolic routing architecture that classifies each incoming query and dispatches it to the cheapest solver capable of producing a correct answer: deterministic symbolic engines for structured tasks and an SLM only for open-ended word problems. Evaluated entirely offline on a Raspberry Pi 4B (8 GB RAM, no GPU) using prompts drawn from three established benchmarks (DeepMind Mathematics, GSM8K, RuleTaker), our best system achieves 99.0% overall accuracy (95% CI: [97.1%, 99.8%]) across 100 prompts (300 trials), compared to 37.3% for the best SLM-only baseline at an equivalent token budget. At matched token budgets, routing provides a 42.4 percentage-point improvement over SLM-only inference (79.7% vs. 37.3% at 30 tokens), and an estimated 11.7-point gain at 512 tokens. Structured categories (arithmetic, algebra, logic) reach 100% accuracy with sub-millisecond latency, while the system median latency drops from 8.6 s to 6 ms (though word problems remain slow at approximately 137 s per prompt). An ablation study across five system versions reveals that each symbolic solver contributes a discrete accuracy gain, and that naive post-processing layers can cause regressions when applied across problem types. A token-budget ablation quantifies the accuracy–energy tradeoff: increasing the generation budget from 30 to 512 tokens raises word-problem accuracy from 18.7% to 96.0% at a cost of 6.9× higher energy per prompt. All experiments, data, and code are publicly available.**

***Impact Statement*—Edge devices such as Raspberry Pi computers increasingly serve as platforms for intelligent applications in healthcare, agriculture, and education—contexts where cloud connectivity may be unavailable or undesirable for privacy reasons. However, small language models running on such devices achieve only 37% accuracy on structured reasoning tasks. Our neurosymbolic routing architecture raises this to 99% by dispatching queries to purpose-built symbolic solvers when appropriate, reserving the language model for genuinely open-ended problems. This approach enables reliable offline reasoning on hardware costing under $100, with three categories of queries resolved in under 12 milliseconds. The architecture is model-agnostic and could be integrated into educational tutoring systems, offline scientific calculators, or field-deployed decision support tools where correctness and low latency are critical.**

***Index Terms*—Edge inference, language model routing, neurosymbolic AI, Raspberry Pi, symbolic reasoning**

I. Introduction

EDGE deployment of language models is motivated by latency, privacy, and connectivity constraints that preclude cloud inference [1]. However, small language models (SLMs) achieve limited accuracy on reasoning tasks when constrained to the CPU-only, memory-limited hardware typical of edge devices such as single-board computers.

Manuscript submitted for review.

Throughout this paper, we use *SLM* (small language model) to refer to models with fewer than 10 billion parameters, consistent with recent usage [2]. All "SLM-only baselines" and "neural inference" refer to the same class of models.

A key observation is that many "reasoning" queries presented to language models are structurally deterministic: arithmetic expressions have unique numeric answers, algebraic equations can be solved symbolically, and formal logic queries admit closed-form evaluation. Routing these structured tasks to purpose-built symbolic solvers—rather than forcing a probabilistic language model to approximate their answers—should yield both higher accuracy and lower latency.

We propose a *neurosymbolic adaptive router* that classifies each query into one of four categories—arithmetic (AR), algebra (ALG), formal logic (LOG), and word problems (WP)—and dispatches it to the appropriate solver:

- **AR** → safe arithmetic evaluator (Python `ast`),
- **ALG** → SymPy symbolic algebra solver,
- **LOG** → forward-chaining logic engine with closed-world assumption,
- **WP** → SLM with chain-of-thought prompting.

Only word problems—which require natural-language comprehension and multi-step reasoning—are routed to the language model. The remaining 75% of queries are handled by deterministic solvers that are both faster and provably correct for their input domains.

We evaluate this architecture on a Raspberry Pi 4B running two quantized SLMs (Phi-4-mini 3.8B, Q6_K; Qwen2.5-Math-1.5B, Q6_K) across 100 prompts drawn from three established benchmarks—DeepMind Mathematics [3], GSM8K [4], and RuleTaker [5]—with three repeats per prompt (300 total trials). Our contributions are:

1) A neurosymbolic routing architecture for edge devices that achieves 99.0% accuracy versus 37.3% for SLM-only baselines, with a 42.4 percentage-point gain at matched token budgets (30 tokens).
2) An ablation across five system versions demonstrating the marginal contribution of each symbolic solver.
3) A token-budget ablation quantifying the accuracy–energy tradeoff for word problems on constrained hardware.
4) A forward-chaining logic engine for RuleTaker-style queries achieving 100% accuracy with < 1 ms latency.
5) Open-source release of all code, data, and experimental logs for reproducibility.

II. Related Work

A. Edge Inference

Deploying language models on resource-constrained devices has been explored through quantization [6], [7], pruning [8], and knowledge distillation [9]. Frameworks such as llama.cpp [10] and Ollama enable CPU-only inference for quantized models on commodity hardware. Our work is complementary: we reduce the *number* of queries routed to the SLM rather than optimizing the model itself.

B. Neurosymbolic AI

Combining neural and symbolic reasoning has a long history [11], [12]. Recent work integrates tool use into SLM pipelines: Toolformer [2] teaches models to call APIs, while PAL [13] generates executable code. Our approach differs in that routing decisions are made *before* the SLM is invoked, using a lightweight regex-based classifier, rather than relying on the model to decide when to use tools.

C. LLM Routing and Cascading

FrugalGPT [14] cascades queries through progressively larger models. Router-based systems like RouteLLM [15] learn to dispatch queries to strong or weak models. Our router operates at a different granularity: rather than choosing between models of different sizes, it chooses between *qualitatively different solvers*—symbolic engines versus neural inference.

D. Mathematical Reasoning

Chain-of-thought prompting [16] and its variants [17] improve mathematical reasoning but remain probabilistic. Tool-augmented approaches [4] combine models with calculators. We extend this paradigm by routing entire categories to symbolic solvers, not just individual arithmetic operations within a generation.

III. System Architecture

A. Overview

The system processes each query q in two stages: (1) a *classifier* $C(q) \in \{\texttt{AR}, \texttt{ALG}, \texttt{LOG}, \texttt{WP}\}$ determines the problem category, and (2) a *dispatcher* routes q to the corresponding solver. The classifier is a deterministic, regex-based function with zero inference cost. The dispatcher invokes one of four solver modules.

1) Solver Naming Convention: Solvers are labeled A1–A7 following the project's internal development order, not the order of introduction in this paper. For clarity:

- **A1**: Fast SLM inference (12 tokens, used as fallback).
- **A2**: Extended SLM inference (30–512 tokens, primary for WP).
- **A3**: Post-processing repair layer (re-prompts SLM with prior context).
- **A4**: SymPy algebra solver (deterministic, for ALG).
- **A5**: Safe arithmetic evaluator (deterministic, for AR).
- **A6**: Forward-chaining logic engine (deterministic, for LOG).

A1 and A3 serve as fallback and repair actions respectively; the primary solvers are A5 (AR), A4 (ALG), A6 (LOG), and A2 (WP).

B. Primary Solvers

- **A5: Arithmetic evaluator.** Extracts numeric expressions from the query using regex, parses them into a Python AST, and evaluates using a whitelist of safe operators (`+`, `-`, `*`, `/`, `**`, `%`, `//`). Only `ast.Constant` and `ast.BinOp` nodes are permitted; exponents are capped at 10,000.
- **A4: Algebra solver.** Extracts equations and target variables, inserts implicit multiplication (e.g., `3x` → `3*x`), and solves via `sympy.solve()`. Supports single- and multi-variable linear systems.
- **A6: Logic engine.** Parses natural-language facts, rules, and queries from RuleTaker-style prompts into structured representations. Applies forward chaining with a closed-world assumption to derive new facts until a fixed point (bounded at 50 iterations), then evaluates the query against the derived fact set. See Section III-E.
- **A2: SLM chain-of-thought.** Constructs a system prompt ("Solve this math problem. Put your final answer in `\boxed{}`"), sends the query to a locally served SLM via llama.cpp HTTP API, and extracts the final numeric answer.

C. Classifier Design

The classifier applies regex rules in priority order:

1) **LOG**: Query matches RuleTaker patterns ("if someone," "if…then," "all…are").
2) **ALG**: Query contains "solve…for [variable]" or has variable–operator–digit patterns with an equals sign.
3) **AR**: A numeric arithmetic expression can be extracted.
4) **WP**: Default fallback for natural-language reasoning.

This ordering ensures that structured queries are intercepted before reaching the SLM. The classifier achieves 100% routing accuracy on our benchmark (all 100 prompts correctly classified across all system versions). It runs in < 10 ms on the Pi and requires no model inference. We note that this perfect accuracy is partly due to the well-separated nature of the benchmark categories; adversarial or ambiguous inputs could cause misclassification (see Section VIII).

D. Fallback Chains

When a primary solver fails (e.g., SymPy cannot parse an equation), the system escalates through a fallback chain:

- AR: A5 → A1 → A2
- ALG: A4 → A1 → A2
- LOG: A6 → A1
- WP: A2 → A3 (repair)

E. A6: Forward-Chaining Logic Engine

The logic engine processes RuleTaker-style prompts through three stages:

1) Parsing: Natural-language statements are converted to structured facts and rules:

- "Felix is a cat" → `fact(felix, is, cat)`
- "All cats are animals" → `rule(?x, is, cat ⇒ ?x, is, animal)`
- "Is Felix an animal?" → `query(felix, is, animal)`

2) Forward Chaining: The engine iteratively applies rules to the current fact set, deriving new facts until no new facts can be produced (fixed-point convergence) or a safety bound of 50 iterations is reached. New facts are added eagerly (immediately available for subsequent rule applications within the same iteration).

3) Query Evaluation: Under the closed-world assumption, a query is answered "Yes" if the queried fact exists in the derived set, and "No" otherwise.

IV. Experimental Setup

A. Hardware

All experiments run on a single Raspberry Pi 4 Model B with 8 GB RAM, a quad-core ARM Cortex-A72 CPU (1.8 GHz), and no GPU or neural accelerator. The operating system is Raspberry Pi OS (64-bit, Debian-based). Models are served via llama.cpp [10] in CPU-only mode with 4 threads.

B. Models

We evaluate two quantized SLMs:

- **Phi-4-mini** (Microsoft, 3.8B parameters, Q6_K quantization, 2.4 GB)
- **Qwen2.5-Math-1.5B** (Alibaba, 1.5B parameters, Q6_K quantization)

All SLM calls use greedy decoding: temperature $= 0.0$, top-$k = 1$, top-$p = 1.0$, seed $= 42$. Generation stops at the first newline character. These settings ensure deterministic, reproducible outputs.

C. Dataset

The benchmark comprises 100 prompts sampled from four established datasets, with full provenance tracking (source dataset, record ID, and field mapping version recorded for every prompt):

- **AR** (25 prompts): From the *DeepMind Mathematics Dataset* [3], modules including addition/subtraction, multiplication, division, and mixed operations.
- **ALG** (25 prompts): From the *DeepMind Mathematics Dataset* [3], modules including linear 1D, linear 2D, and polynomial roots.
- **LOG** (25 prompts): From *RuleTaker* [5], forward-chaining logical entailment queries over natural-language facts and rules.
- **WP** (25 prompts): From *GSM8K* [4], multi-step grade-school word problems requiring natural-language comprehension.

Prompts were sampled with a fixed global seed (42) using stratified random sampling from the original dataset splits. No prompts were paraphrased, generated, or modified beyond whitespace normalization. Cross-tier disjointness was verified by both source key and text hash.

The benchmark is intentionally small (100 prompts, 300 trials) due to the slow inference speed on the Pi (approximately 2 minutes per word problem at 512 tokens). We acknowledge this limits statistical power: the 99.0% accuracy has a 95% Wilson confidence interval of [97.1%, 99.8%]. Furthermore, the 3 repeats are primarily useful for measuring SLM variance on WP; the symbolic solvers are deterministic, so repeats add no information for AR, ALG, and LOG (effective $n = 25$ per category). Larger-scale evaluation is an important direction for future work.

Each prompt is run 3 times per system configuration, yielding 300 trials per system. Evaluation uses *exact match*: a trial is correct if and only if the extracted numeric answer (or yes/no for LOG) matches the ground truth.

D. Metrics

- **Accuracy**: Fraction of trials with exact-match correct answers.
- **Latency**: Wall-clock time per inference, measured from prompt submission to answer extraction (millisecond precision).
- **Energy**: Total run energy measured via USB power meter (start/end mWh readings per run), divided by number of prompts.

E. Error Taxonomy

We classify errors by category and failure mode. Error codes are assigned per category (gaps in numbering reflect unused codes):

- **E0**: Correct answer.
- **E1**: Arithmetic error (wrong numeric result on AR).
- **E2**: Logic error (wrong yes/no on LOG).
- **E3**: Algebra error (wrong numeric result on ALG).
- **E5**: Word problem error (wrong numeric result on WP).
- **E7**: Timeout (model exceeded time limit before producing output).
- **E8**: Parse failure (could not extract a valid answer from model output).

V. System Versions and Ablation

We developed the system incrementally across five versions, each adding or modifying a single component. This progression serves as a natural ablation study.

A. Baseline: SLM-Only

All 100 prompts are sent directly to the SLM with no routing or symbolic solvers. We tested four baseline configurations varying two factors:

- **Token budget**: C1 uses A1 (12 tokens); C2 uses A2 (30 tokens).
- **Grammar constraints**: *grammar* constrains the output format via a BNF grammar; *nogrammar* allows free-form generation.

The best baseline (C2 with grammar, Phi-4-mini) achieves 37.3% overall accuracy: 48% on AR, 40% on ALG, 53% on LOG, and 8% on WP. Parse failures (E8) account for 17% of all trials.

B. V1: Add Arithmetic Solver (+A5)

AR queries are routed to the safe arithmetic evaluator.

Result: AR accuracy jumps from 48% to **100%**, latency drops from 6.2 s to **1 ms**. Overall accuracy rises to 53.0%.

C. V2: Add Algebra Solver (+A4)

ALG queries are additionally routed to SymPy.

Result: ALG accuracy jumps from 40% to **96–100%**, overall accuracy reaches 67.0%. ALG latency drops from 5.8 s to **11 ms**.

D. V3: Add Post-Processing Repair Layer (+A3)

An answer-repair module (A3) attempts to extract and correct answers from verbose SLM outputs. In V3.0, this was applied to *both* WP and LOG.

Result: WP accuracy improves modestly (8% $\rightarrow$ 18.7%). However, applying A3 to LOG causes a regression: LOG accuracy *drops* from 64% to 36.7%. The repair layer, designed for numeric extraction, corrupts yes/no logic answers—e.g., by extracting spurious numbers from reasoning text.

V3.1 fix: Disabling A3 for LOG restores LOG accuracy to 64%. Overall: 70.7%.

Observation: This is an engineering error (applying a numeric extractor to yes/no outputs) rather than a deep methodological insight, but it illustrates that post-processing layers are format-sensitive and should not be applied across output types without validation.

E. V4: Add Confidence Calibration

A logistic regression calibrator estimates $P(\text{correct})$ for each SLM response based on features (response length, parse success, timeout status). If $P(\text{correct}) < \tau = 0.70$, the response is escalated to A3 repair.

Result: 70.7% accuracy—identical to V3.1. The calibrator correctly flags low-confidence answers, but the repair layer cannot produce correct alternatives from truncated 30-token responses. In retrospect, this is expected: if the model lacks sufficient tokens to reason through a problem, identifying the answer as probably wrong does not produce a correct one. The correct intervention is to remove the bottleneck (increase token budget), not to add detection around it.

F. V5: Add Logic Engine + Increase Token Budget (+A6, +CoT)

Two changes address the remaining bottlenecks:

1) **A6 forward-chaining logic engine** replaces SLM inference for LOG, achieving 100% accuracy with < 1 ms latency.
2) **Chain-of-thought prompting with 512 tokens** replaces the 30-token budget for WP, with the system prompt: "Solve this math problem. Put your final answer in `\boxed{}`."

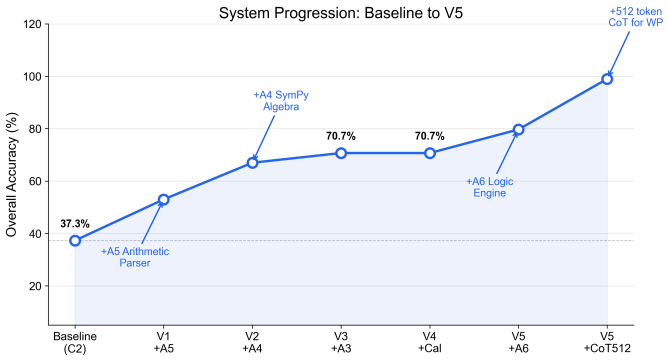

Fig. 1. System progression from baseline (37.3%) through V5 (99.0%), showing the accuracy gain from each added component. The plateau at V3–V4 (70.7%) reflects the token-budget bottleneck that V5 resolves.

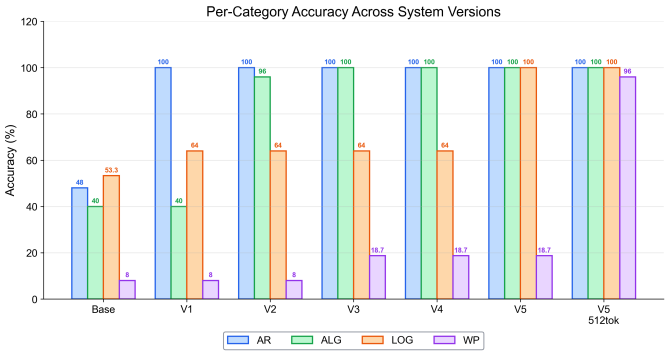

Fig. 2. Per-category accuracy across system versions. Each symbolic solver produces a discrete jump in its target category: A5 for AR (V1), A4 for ALG (V2), A6 for LOG (V5). WP accuracy remains low until the token budget is increased in V5.

Result: 99.0% overall accuracy. AR, ALG, and LOG all achieve 100%. WP reaches 96% at 512 tokens. The single remaining failure is analyzed in Section VII-B.

Fig. 1 shows the full system progression from baseline through V5, and Fig. 2 breaks this down by category.

VI. Results

A. Main Results

Table I summarizes the key systems. The neurosymbolic router (V5, Phi-4-mini, 512 tokens) achieves 99.0% overall accuracy versus 37.3% for the best SLM-only baseline.

A note on the baseline comparison. The 37.3% baseline uses a 30-token budget, which is restrictive. Part of V5's improvement comes from the increased token budget (30 $\rightarrow$ 512), not only from routing. To isolate the routing contribution, we note that V5 at 30 tokens ("V5 Default" in Table I) achieves 79.7%—already a 42.4 percentage-point gain over baseline at the same token budget, entirely due to symbolic routing of AR, ALG, and LOG. We did not run a pure SLM-only baseline at 512 tokens; however, we can estimate an upper bound. Assuming optimistically that the SLM achieves 100% on AR

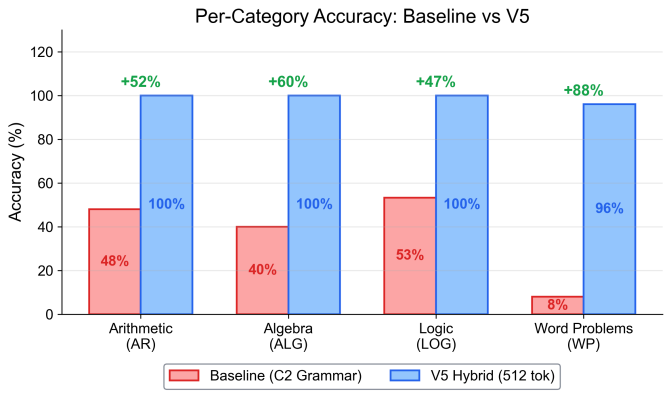

Fig. 3. Per-category accuracy: baseline (C2) vs. V5 (512 tokens). Green annotations show the percentage-point gain per category. AR, ALG, and LOG reach 100% via symbolic solvers; WP improves from 8% to 96% via chain-of-thought prompting.

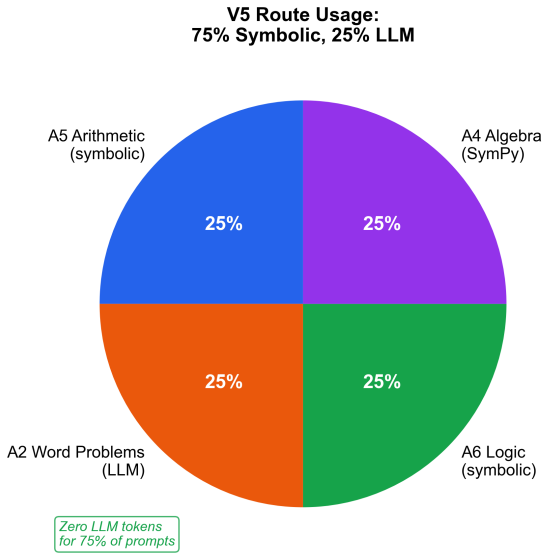

Fig. 4. Route distribution in V5: 75% of queries are dispatched to deterministic symbolic solvers (A5, A4, A6), with only 25% requiring SLM inference (A2 for WP).

and ALG at 512 tokens, retains its baseline LOG accuracy of 53%, and matches V5's WP accuracy of 96%, the ceiling is $0.25 \times 1.0 + 0.25 \times 1.0 + 0.25 \times 0.53 + 0.25 \times 0.96 = 87.3\%$—still 11.7 percentage points below V5's 99.0%.

B. Per-Category Accuracy

Table II and Fig. 3 show accuracy by problem category. Symbolic solvers achieve perfect accuracy on their respective domains.

C. Latency Analysis

Table III shows median latency by category for V5 (512 tokens) versus the best baseline. Symbolic solvers achieve three orders-of-magnitude speedups on structured categories:

- AR: **1 ms** (vs. 6.3 s baseline, 6,300× speedup)
- ALG: **11 ms** (vs. 5.8 s baseline, 530× speedup)

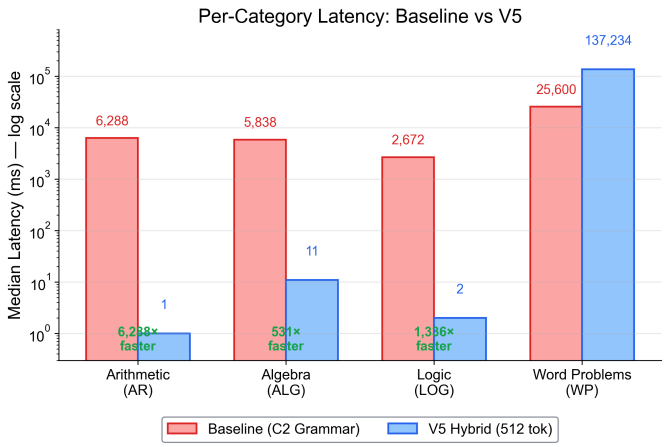

Fig. 5. Per-category median latency on a log scale: baseline vs. V5. Symbolic solvers achieve 530–6,300× speedups on structured categories. WP latency increases due to the higher token budget.

- LOG: **2 ms** (vs. 2.7 s baseline, 1,350× speedup)
- WP: **137 s** (increased from 25 s due to higher token budget)

The system-wide median of 6 ms reflects the dominance of symbolic routes (75% of queries). However, the WP latency of approximately 137 s per prompt is a significant practical limitation for real-time applications. Fig. 5 visualizes this on a log scale.

D. Energy–Accuracy Tradeoff

Table IV shows the effect of token budget on word-problem accuracy and energy. The tradeoff is substantial: 512 tokens yields 96% WP accuracy but costs 6.9× more energy per prompt than 30 tokens.

Note on identical "V5 Default" numbers for Phi and Qwen. Both models show 18.7% WP accuracy at 30 tokens with identical latency and energy. This is because V5 Default routes 75% of queries to deterministic solvers (identical results regardless of model), and the remaining WP queries at 30 tokens mostly time out or parse-fail—the model barely generates useful output, so both models fail on the same prompts. At higher token budgets where the models can actually reason, performance diverges: Phi achieves 92% at 300 tokens versus Qwen's 64%.

Note that the best system (V5, Phi, 512tok) uses *more* energy per prompt (211.1 mWh) than the baseline (75.0 mWh). The energy savings from symbolic routing on AR/ALG/LOG are outweighed by the increased token budget for WP. This tradeoff—higher accuracy at higher energy—is fundamental and is discussed further in Section VIII. Fig. 7 compares energy consumption across all system configurations.

E. Error Distribution

Table V shows the error taxonomy for selected systems. V5 at 512 tokens eliminates all error types except E5 (3 word-problem errors out of 300 trials), corresponding to a single prompt that fails consistently across all 3 repeats.

TABLE I
MAIN RESULTS (300 TRIALS PER SYSTEM)

System	Acc. (%)	Med. Latency (ms)	Energy (mWh/p)	Parse Fail (%)
Baseline (C2, grammar)	37.3	8,612	75.0	17.0
Baseline (C2, no grammar)	37.0	8,709	64.6	17.0
Baseline (C1, grammar)	36.0	6,021	32.1	26.0
Baseline (C1, no grammar)	33.0	9,080	59.9	26.0
Hybrid V1 (+A5)	53.0	4,147	75.9	15.0
Hybrid V2 (+A4)	67.0	2,598	37.2	15.0
Hybrid V3.1 (+A3)	70.7	1,413	34.0	4.3
Hybrid V4 (+Calibration)	70.7	1,466	34.0	4.0
V5 Default (30 tokens)	79.7	6	30.6	4.0
V5 (Phi, 300 tokens)	98.0	6	147.2	0.0
V5 (Phi, 512 tokens)	**99.0**	**6**	211.1	**0.0**
V5 (Qwen, 512 tokens)	**99.0**	6	147.6	0.0

TABLE II
PER-CATEGORY ACCURACY (%) ACROSS KEY SYSTEMS

System	AR	ALG	LOG	WP	All
Baseline (best)	48.0	40.0	53.3	8.0	37.3
V1 (+A5)	100	40.0	64.0	8.0	53.0
V2 (+A4)	100	96.0	64.0	8.0	67.0
V3.1 (+A3)	100	100	64.0	18.7	70.7
V5 Default	100	100	100	18.7	79.7
V5 (512tok)	100	100	100	96.0	99.0

TABLE III
MEDIAN LATENCY BY CATEGORY

System	AR (ms)	ALG (ms)	LOG (ms)	WP (s)
Baseline (best)	6,288	5,838	2,672	25.6
V5 (512tok)	1	11	2	137.2

TABLE IV
WORD-PROBLEM ACCURACY VS. TOKEN BUDGET

Model	Tok.	CoT	WP (%)	Lat. (s)	E (mWh/p)
Phi-4-mini	30	No	18.7	26.0	30.6
Phi-4-mini	300	Yes	92.0	128.6	147.2
Phi-4-mini	512	Yes	96.0	135.2	211.1
Qwen-1.5B	30	No	18.7	26.0	30.6
Qwen-1.5B	300	Yes	64.0	100.1	113.1
Qwen-1.5B	512	Yes	96.0	128.2	147.6

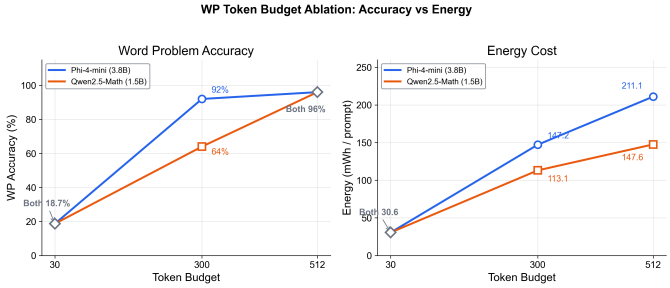

Fig. 6. Word-problem token budget ablation for two SLMs: accuracy (left) and energy cost (right) as a function of token budget. Both models converge to 96% at 512 tokens, but Phi-4-mini is more token-efficient at 300 tokens.

VII. ANALYSIS

A. Why Symbolic Routing Works

The accuracy gains from symbolic routing are not incremental: each solver produces a *discrete jump* in its category. Adding A5 moves AR from 48% to 100%; adding A4 moves ALG from 40% to 96–100%; adding A6 moves LOG from 64% to 100%. These reflect the replacement of probabilistic approximation with deterministic computation.

This suggests a design principle: *when the structure of a problem is known, symbolic solvers dominate neural inference in both accuracy and efficiency.* The SLM's value lies in handling queries whose structure is not known in advance. Fig. 8 visualizes the category-level accuracy across all system versions and configurations tested.

B. Error Analysis: The Single V5 Failure

The sole error in V5 (Phi-4-mini, 512 tokens) occurs on prompt WP_000112 from GSM8K:

> *"At Allan's house, there is twice as much corn as cannolis. He has a total of 40 cannolis. Allan bought 60 more cannolis at the store and 40 fewer corns than the number of cannolis. Find the combined total of the number of corns and cannolis Allan has in the house?"*

Expected answer: **200**. Both models consistently produce **240**. The chain-of-thought output reveals the error:

> *"Allan bought 60 more cannolis, so now he has* $40+60 = 100$ *cannolis. He also bought 40 fewer corns than the number of cannolis he bought, which means he bought* $100 - 40 = 60$ *corns."*

The model interprets "40 fewer corns than the number of cannolis" as referring to the *updated* cannoli count (100) rather than the original count (40), yielding $100 - 40 = 60$ new corns instead of the intended $40 - 40 = 0$. This is a linguistic ambiguity in the problem statement: the referent of "the number of cannolis" is genuinely unclear. Both Phi-4-mini and Qwen2.5-Math produce the same interpretation across all trials, suggesting this is a systematic parsing bias rather than a stochastic error.

C. Model Comparison

At 512 tokens, both models achieve 99.0% overall and 96.0% WP accuracy. However, at 300 tokens, Phi-4-mini achieves 92.0% WP accuracy versus Qwen's 64.0%, suggesting Phi-4-mini is more token-efficient for word-problem reasoning despite Qwen's mathematics-specific pretraining.

Both models achieve identical performance on symbolic categories (100%) because these queries never reach the model.

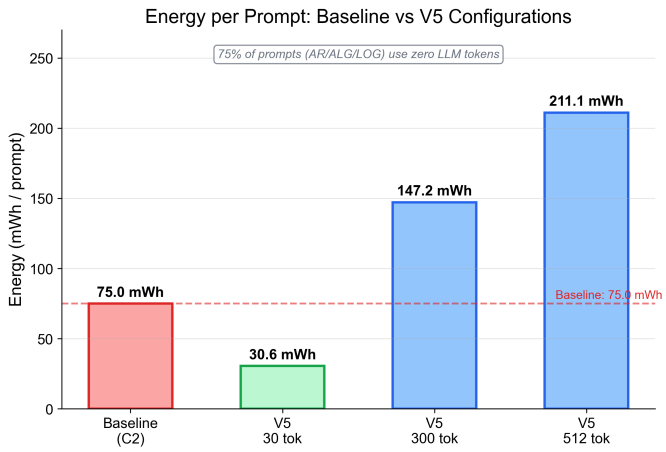

Fig. 7. Energy per prompt (mWh) across system configurations. Higher token budgets (300, 512) dramatically increase energy cost. V5 Default (30 tokens) achieves the lowest energy among hybrid systems.

TABLE V
ERROR DISTRIBUTION (300 TRIALS EACH). E0 = CORRECT

System	E0	E1	E2	E3	E5	E7	E8
Baseline	112	36	26	45	21	9	51
V1 (+A5)	159	0	27	45	24	0	45
V2 (+A4)	201	0	27	3	24	0	45
V3.1 (+A3)	212	0	27	0	18	30	13
V5 (512)	297	0	0	0	3	0	0

VIII. DISCUSSION

A. Generalizability

Our benchmark covers four well-defined categories drawn from established datasets. Real-world queries may span additional categories (e.g., commonsense reasoning, spatial reasoning) that are neither fully symbolic nor standard word problems. Extending the router to handle such cases—perhaps with a learned classifier—is an important direction.

B. Classifier Robustness

The regex classifier achieves 100% accuracy on our benchmark, but this reflects the well-separated nature of the benchmark categories rather than adversarial robustness. Misclassification would route a query to the wrong solver, potentially yielding an incorrect answer with high confidence. A learned classifier or a fallback mechanism that detects solver failures and re-routes would improve robustness.

C. Dataset Size

Our benchmark of 100 prompts (25 per category) is small by modern standards. The 95% confidence interval on 99.0% accuracy is [97.1%, 99.8%], which is reasonably tight, but per-category confidence intervals are wider (e.g., 96% WP accuracy on 25 prompts has a 95% CI of [80.5%, 99.3%]). Evaluation on larger subsets of the source benchmarks would strengthen the claims.

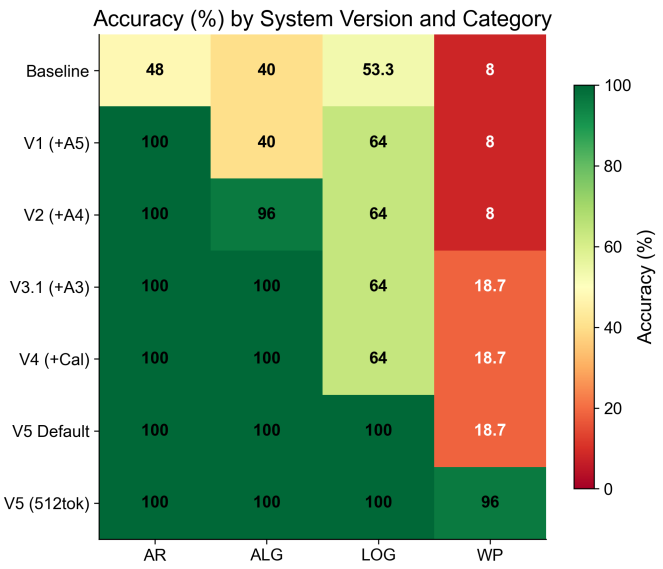

Fig. 8. Accuracy by category across all system versions and configurations. Each column is a category; each row is a system version. Symbolic solvers (A5, A4, A6) produce immediate 100% accuracy in their target categories upon introduction.

D. Energy–Accuracy Tradeoff

The best-accuracy system (V5, 512tok) uses 2.8× more energy than the baseline. The energy savings from symbolic routing on 75% of queries are real but are outweighed by the 17× increase in WP generation cost. For battery-powered deployments, adaptive token budgets—starting low and escalating only when the model's output indicates insufficient reasoning—could reduce average energy while maintaining accuracy.

E. Scalability of Symbolic Solvers

Our solvers handle single-variable linear equations, basic arithmetic, and propositional logic. Extending to nonlinear equations, multi-variable systems, or first-order logic would require more powerful solvers, but the routing architecture would remain unchanged.

F. Practical WP Latency

Word problems take approximately 137 s per prompt at 512 tokens on the Pi 4B's CPU. This is acceptable for batch processing or non-interactive applications but prohibitive for real-time use. Faster hardware (e.g., with neural accelerators) or more aggressive quantization could reduce this.

IX. CONCLUSION

We presented a neurosymbolic routing architecture for edge devices that achieves 99.0% accuracy on a 100-prompt reasoning benchmark drawn from DeepMind Mathematics, GSM8K, and RuleTaker, compared to 37.3% for SLM-only baselines at 30 tokens and an estimated 87.3% optimistic ceiling for SLM-only at 512 tokens. The system runs entirely offline on a Raspberry Pi 4B with no GPU.

Our ablation demonstrates that each symbolic solver produces a discrete accuracy jump in its category. The system's remaining weakness is word problems, where accuracy depends on token budget and thus energy: 96% WP accuracy at 512 tokens costs 6.9× more energy than 30 tokens. Future work will explore adaptive token allocation and extending symbolic solver coverage. More broadly, this work demonstrates that trustworthy reasoning on edge devices does not require larger models or cloud connectivity—it requires routing each query to the right solver.

REFERENCES

[1] Z. Xu, Y. Xu, J. Li *et al.*, "A survey on efficient inference for large language models," *arXiv preprint arXiv:2404.14294*, 2024.

[2] T. Schick, J. Dwivedi-Yu, R. Dessì *et al.*, "Toolformer: Language models can teach themselves to use tools," *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[3] D. Saxton, E. Grefenstette, F. Hill, and P. Kohli, "Analysing mathematical reasoning abilities of neural models," in *International Conference on Learning Representations*, 2019.

[4] K. Cobbe, V. Kosaraju, M. Bavarian *et al.*, "Training verifiers to solve math word problems," *arXiv preprint arXiv:2110.14168*, 2021.

[5] P. Clark, O. Tafjord, and K. Richardson, "Transformers as soft reasoners over language," in *Proceedings of the International Joint Conference on Artificial Intelligence*, 2020.

[6] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit matrix multiplication for transformers at scale," *Advances in Neural Information Processing Systems*, vol. 35, 2022.

[7] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate post-training quantization for generative pre-trained transformers," *arXiv preprint arXiv:2210.17323*, 2023.

[8] M. Sun, Z. Liu, A. Bair, and J. Z. Kolter, "A simple and effective pruning approach for large language models," *arXiv preprint arXiv:2306.11695*, 2024.

[9] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.

[10] G. Gerganov, "llama.cpp: LLM inference in pure C/C++," https://github.com/ggerganov/llama.cpp, 2023, accessed: 2026-03-01.

[11] A. d. Garcez and L. C. Lamb, "Neural-symbolic computing: An effective methodology for principled integration of machine learning and reasoning," *Journal of Applied Logics*, vol. 6, no. 4, pp. 611–632, 2019.

[12] H. Kautz, "The third AI summer: AAAI robert s. engelmore memorial lecture," *AI Magazine*, vol. 43, no. 1, pp. 93–104, 2022.

[13] L. Gao, A. Madaan, S. Zhou *et al.*, "PAL: Program-aided language models," in *Proceedings of the 40th International Conference on Machine Learning*, 2023.

[14] L. Chen, M. Zaharia, and J. Zou, "FrugalGPT: How to use large language models while reducing cost and improving performance," *arXiv preprint arXiv:2305.05176*, 2023.

[15] I. Ong, A. Almahairi, V. Wu *et al.*, "RouteLLM: Learning to route LLMs with preference data," *arXiv preprint arXiv:2406.18665*, 2024.

[16] J. Wei, X. Wang, D. Schuurmans *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in Neural Information Processing Systems*, vol. 35, 2022.

[17] X. Wang, J. Wei, D. Schuurmans *et al.*, "Self-consistency improves chain of thought reasoning in language models," *arXiv preprint arXiv:2203.11171*, 2023.